

DOCUMENTACIÓN DEL DIRECTOR

Trabajar con Agentes de IA: Lo que Z.ai no te cuenta y cómo resolverlo

Una explicación en lenguaje claro de cómo funciona la plataforma Z.ai, por qué necesitamos una estrategia de actuación, y cómo convertimos cada agente nuevo en un colaborador que ya sabe trabajar con nosotros.

VERSIÓN 3.0 · 2025

PRÓLOGO

El origen de este documento

Este documento nació de una necesidad muy concreta: trabajar con agentes de inteligencia artificial de forma profesional, sin perder tiempo, sin repetir instrucciones, y sin que cada sesión sea como empezar de cero.

La plataforma Z.ai ofrece una capacidad extraordinaria: poner a un agente de IA a programar por vos en un entorno de desarrollo completo. Pero no te da las herramientas para que ese agente se comporte de forma consistente entre sesiones. Te da el motor, pero no el volante.

Lo que sigue es el resultado de varias semanas de trabajo, prueba y error, y refactorización constante. No es un documento técnico. Es una explicación en lenguaje claro de cómo funciona Z.ai, qué puede y qué no puede hacer, y cómo construyó una estrategia que convierte cada agente nuevo en un colaborador que ya sabe trabajar con nosotros.

PARTE I

Qué es Z.ai y cómo funciona

Capítulo 1

La plataforma

Z.ai es una plataforma que permite interactuar con modelos de lenguaje avanzados, llamados GLM, a través de una interfaz web de chat o mediante integración directa con herramientas de desarrollo como Claude Code, Cline, Kilo Code y otras.

La propuesta central de Z.ai es el **GLM Coding Plan**: un paquete de suscripción diseñado específicamente para programar con IA. En lugar de usar el chat como un simple pregunta-respuesta, Z.ai habilita algo más ambicioso: **agentes de código** que pueden leer tu proyecto, modificar archivos, ejecutar comandos, y completar tareas complejas de desarrollo a través de múltiples pasos.

Z.ai no es un simple chatbot con respuestas inteligentes. Es un entorno donde un modelo de IA opera como un programador virtual que tiene acceso a herramientas

reales de desarrollo: puede navegar tu sistema de archivos, escribir código, ejecutarlo en una terminal, buscar en internet, e incluso clonar repositorios de GitHub.

La plataforma ofrece una familia de modelos GLM con diferentes capacidades y precios. El más potente, GLM-5.3, puede manejar tareas complejas de larga duración con un contexto de hasta un millón de tokens, equivalente a cientos de páginas de texto. También existen modelos gratuitos como GLM-4.7-Flash para tareas más ligeras, y modelos especializados en visión, generación de imágenes y video.

> **En resumen:** Z.ai es una plataforma de agentes de código que permite a modelos de IA avanzados operar en entornos de desarrollo reales, con acceso a lectura y escritura de archivos, ejecución de comandos, y herramientas externas.

Capítulo 2

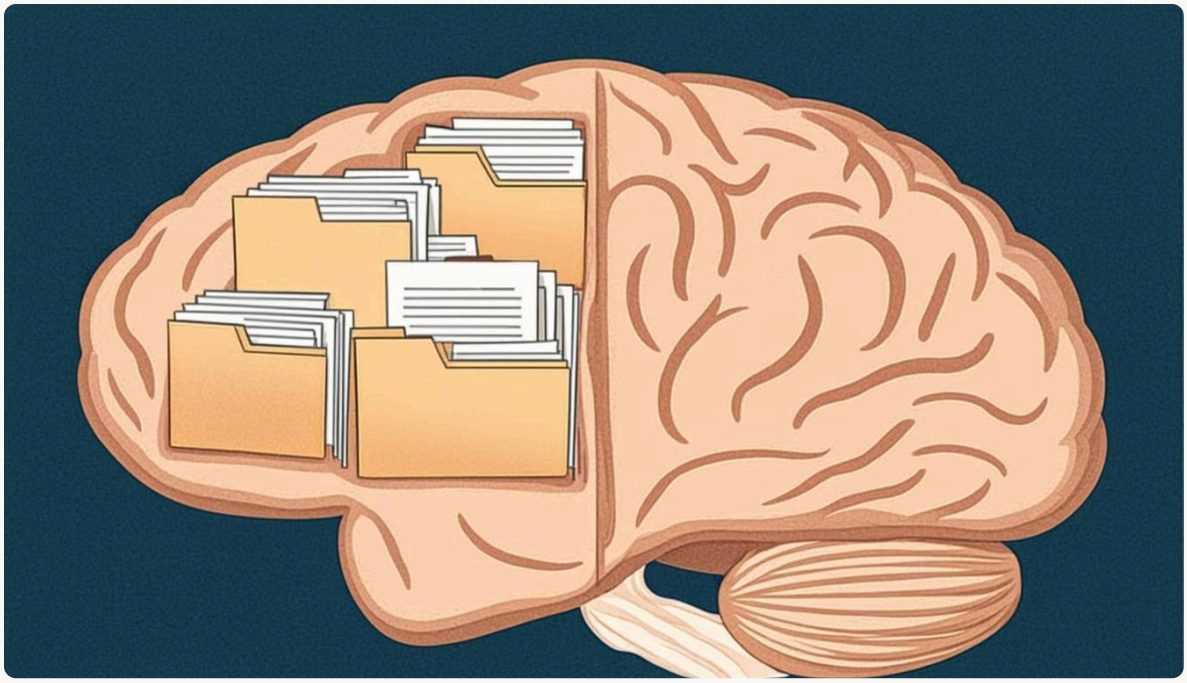
El entorno de trabajo y sus limitaciones

Cuando abrís un agente en Z.ai web, la plataforma te da un **sandbox**: un entorno de desarrollo aislado con un sistema Linux, acceso a la terminal, y un sistema de archivos organizado. El agente puede leer y escribir archivos, ejecutar comandos, y trabajar como si fuera un desarrollador sentado frente a una computadora. Tiene acceso a internet, puede buscar información, clonar repositorios e instalar paquetes.

Pero el sandbox tiene **limitaciones severas** que son el origen de toda esta estrategia. La primera y más importante: **no hay persistencia entre sesiones**. Cuando cerrás un agente y abrís otro, el nuevo agente empieza con un entorno limpio. No hay archivos, no hay memoria, no hay contexto del agente anterior. Es como si cada mañana te contrataras un programador nuevo y excelente, pero que nunca leyó el manual de tu empresa.

La segunda limitación es que **no se puede personalizar el System Prompt**. Z.ai inyecta automáticamente sus propias instrucciones al agente, y no hay forma de que el usuario añada las suyas de forma persistente. A diferencia de otras herramientas como Claude Code, que leen automáticamente un archivo de configuración al iniciar, Z.ai web no carga ningún archivo personalizado.

La tercera limitación es que el worklog, el único archivo que la plataforma le indica al agente que lea, **no persiste entre agentes**. Cada agente nuevo lo encuentra vacío. No hay transferencia automática de información de un agente al siguiente.



La falta de memoria entre sesiones es el problema central que motiva toda la estrategia.

> **En resumen:** El sandbox de Z.ai es un entorno de desarrollo completo pero efímero. Cada agente nuevo empieza de cero, sin memoria del anterior, sin configuración personalizada, y sin forma de cargar automáticamente instrucciones. El worklog, que sería el vehículo ideal para la persistencia, tampoco se transfiere entre sesiones.

PARTE II

La documentación de Z.ai y lo que no cubre

Capítulo 3

Lo que dice la documentación oficial

Z.ai mantiene una documentación extensa que cubre modelos, precios, capacidades, integraciones con herramientas de desarrollo, y una sección llamada **Devpack Resources** que habla de memoria y mejores prácticas para agentes. La documentación es buena para entender qué ofrece la plataforma, pero tiene un vacío importante: está escrita pensando en desarrolladores que integran Z.ai a través de su API o de herram-

ientas como Claude Code, no para quienes usan la interfaz web de chat con sandbox.

Capítulo 4

El mecanismo de memoria

La documentación de Z.ai dedica una página completa al mecanismo de memoria, y es quizás el documento más importante para entender por qué necesitamos una estrategia. La documentación explica que los modelos de lenguaje tradicionales **no preservan estado entre llamadas**. No recuerdan lo que pasó antes. Cada interacción es independiente.

Para resolver esto, los sistemas de agentes usan **memoria externa**: archivos, bases de datos, o vectores que almacenan información entre sesiones. La documentación describe cinco tipos de memoria en un agente de código:

- **Memoria de sesión** — Toda la información de la conversación actual. Es temporal y se pierde cuando la sesión termina.
- **Memoria de proyecto** — Información duradera sobre el código: arquitectura, estándares, flujos de trabajo. Se almacena en archivos que se leen al inicio de cada sesión.
- **Memoria semántica** — Conocimiento factual: documentación de APIs, reglas de lenguajes de programación.
- **Memoria episódica** — Experiencias pasadas del agente: cómo se resolvió un bug anterior, qué estrategia funcionó.
- **Memoria de procedimiento** — Flujos de trabajo y estrategias paso a paso: cómo hacer pruebas, cómo desplegar.

La documentación hace una distinción crucial: separar la **memoria de instrucciones** de la **memoria de aprendizaje**. La memoria de instrucciones la escribe el humano y son las reglas, políticas y restricciones de comportamiento. Deben ser estables y predecibles. La memoria de aprendizaje la construye el agente con la experiencia: preferencias del usuario, errores que se cometieron, técnicas que funcionaron. Si se mezclan, el comportamiento del agente se degrada.

> **En resumen:** La propia documentación de Z.ai reconoce que los agentes necesitan memoria externa y recomienda separar las instrucciones humanas (estables) del aprendizaje del agente (evolutivo). Esta separación es exactamente lo que implementamos en nuestra estrategia.

Capítulo 5

Las mejores prácticas

Otra página clave de la documentación es la de mejores prácticas, que describe cómo trabajar eficientemente con agentes de código. De las prácticas que enumera, las más relevantes para nuestra estrategia son:

- **Tratar al agente como un colaborador, no como un asistente desechable.** No preguntar algo, recibir código y terminar. Un agente es un colaborador configurable que se mejora con el tiempo.
- **Estructurar las entradas de tarea.** Lo que importa más no es la técnica de redacción del prompt, sino el contexto de la tarea: qué se necesita, qué archivos están involucrados, cómo se verifica que se completó.
- **Planificar antes de ejecutar.** Para tareas complejas, pedir al agente que produzca un plan antes de escribir código.
- **Capturar reglas repetitivas en archivos.** Si repetís las mismas instrucciones en cada prompt, esas instrucciones deberían estar en archivos que el agente carga automáticamente.
- **El entorno define lo que el agente puede hacer.** Muchos problemas que parecen ser limitaciones del modelo son en realidad problemas del entorno.

Capítulo 6

El hueco entre la documentación y la realidad

Hay un hueco significativo entre lo que la documentación describe y lo que la interfaz web permite. La documentación asume que estás usando Z.ai a través de Claude Code u otra herramienta que sí tiene mecanismos de persistencia. Pero si usás Z.ai web directamente, el chat con sandbox, **estás solo**.

Lo que la documentación recomienda	Lo que la interfaz web permite
Cargar archivos de proyecto automáticamente al iniciar	No hay mecanismo para esto
System Prompt personalizado y persistente	No se puede personalizar
Memoria de proyecto en archivos como CLAUDE.md	No hay equivalente en Z.ai web
Skills configurables para automatizar flujos	No disponibles en Z.ai web
Persistencia del contexto entre sesiones	No existe: cada agente empieza de cero

> **En resumen:** La documentación de Z.ai describe un mundo ideal donde los agentes tienen memoria persistente, archivos de configuración automáticos, y entorno personalizado. La interfaz web no ofrece ninguno de estos mecanismos. Esa brecha es exactamente lo que nuestra estrategia viene a cerrar.

PARTE III

Por qué necesitamos una estrategia

Capítulo 7

El problema real

Imaginá esta situación, que probablemente te resulte familiar. Abrís un agente nuevo en Z.ai. El agente es capaz, inteligente, y tiene acceso a todas las herramientas de desarrollo. Le das tu tarea. El agente la ejecuta. Todo bien.

Al día siguiente, abrís otro agente para continuar el trabajo. Y ahí empiezan los problemas. El agente no sabe quién sos. No sabe que preferís que te hablen en lenguaje natural, sin tecnicismos. No conoce las reglas de tu proyecto. No sabe que no puede reescribir un archivo completo cuando bastan cambios pequeños. No sabe qué se hizo en la sesión anterior. No tiene contexto del trabajo previo. No sabe qué errores se cometieron antes y cómo evitarlos. No respeta tu flujo de trabajo.

Cada sesión es como contratar a un programador nuevo y excelente, pero que no ha leído el manual de tu empresa. Capaz, pero desalineado. La tentación es pegar un

prompt largo al inicio de cada sesión con todas las reglas, el contexto del proyecto, y las instrucciones de comportamiento. Pero eso tiene tres problemas: es tedioso, es frágil, y es desperdicio. Las reglas de comportamiento son siempre las mismas, el contexto del proyecto evoluciona, y mezclar ambos en un prompt manual es ineficiente.

Capítulo 8

Lo que intentamos antes

No llegamos a la solución actual a la primera. Pasamos por varias iteraciones, cada una un poco mejor que la anterior.

VERSIÓN 1 — EL PROMPT MONOLÍTICO

Al principio, pegábamos todas las instrucciones en un solo prompt enorme al inicio de cada sesión. Funcionaba parcialmente, pero era insostenible: demasiado largo, demasiado manual, y el agente a veces ignoraba partes por la cantidad de información.

VERSIÓN 2 — ARCHIVOS MODULARES

Separamos las instrucciones en varios archivos (identidad, contrato, comunicación, protocolo de trabajo, entorno) y el prompt inicial se convirtió en un índice que le decía al agente qué archivos leer. Mejor, pero seguía requiriendo que el Director pegara ese índice cada vez.

VERSIÓN 2.1 — REPOSITORIO DE GITHUB

Subimos los archivos a un repositorio privado y el prompt indicaba al agente que lo clonara. Un paso adelante, pero el prompt seguía siendo manual y teníamos que pegar instrucciones de arranque.

VERSIÓN 3 — EL WORKLOG COMO BOOTSTRAP

Descubrimos que Z.ai obliga a cada agente a leer un archivo llamado worklog.md al iniciar. Ese es el único archivo que se garantiza que el agente lea. Si ponemos las instrucciones de estrategia ahí, el agente las lee automáticamente. El worklog es como la pizarra de la entrada de la oficina: no necesitás decirle al empleado que la lea, ya está ahí cuando entra.

En resumen: La necesidad de una estrategia surge de un problema concreto: cada agente nuevo en Z.ai empieza de cero, sin reglas, sin contexto, sin memoria. Las soluciones manuales no son sostenibles. La solución es aprovechar el único mecanismo automático que Z.ai ofrece, la lectura del worklog, para inyectar toda la configuración del agente sin intervención manual.

PARTE IV

La estrategia v3.0

Capítulo 9

La idea en una frase

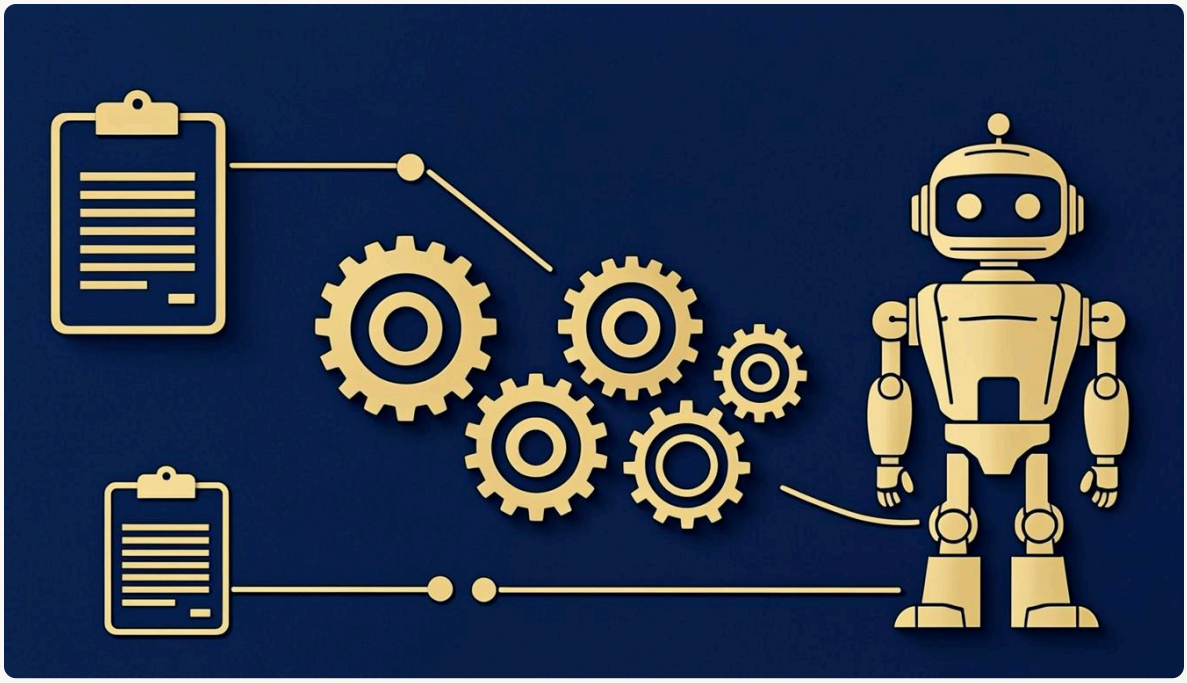
> El worklog es el bootstrap. Los archivos en el repositorio son el contrato. El prompt inicial es una verificación.

Todo lo que sigue es desarrollo de esa idea. La estrategia se basa en una cadena de eventos que se ejecuta automáticamente cuando se configura correctamente, convirtiendo un simple copia y pega inicial en un proceso de configuración completa.

Capítulo 10

La cadena de arranque

Cuando se configura correctamente, la sesión con un agente sigue una secuencia automática que va desde la acción más simple del Director hasta la configuración completa del agente. Veamos paso a paso cómo funciona.



La cadena de arranque: del copia y pega del Director a la configuración automática del agente.

Paso uno: el Director pega el worklog. Al abrir un agente nuevo, el Director copia el contenido del worklog de la sesión anterior, o una plantilla si es la primera vez, y lo pega en el worklog del nuevo agente. Es la única acción manual. Un copia y pega. Esta transferencia es necesaria porque el worklog no persiste entre agentes: cada nuevo agente lo encuentra vacío.

Paso dos: Z.ai lee el worklog. El System Prompt de Z.ai, las instrucciones internas de la plataforma, le indica al agente que lea el archivo worklog.md. El agente lo hace automáticamente, sin que nadie se lo pida.

Paso tres: el agente encuentra las instrucciones. Arriba del todo del worklog, antes de cualquier entrada de sesión, hay una sección de estrategia que le dice al agente qué hacer: clonar un repositorio, leer unos archivos, y emitir una frase específica.

Paso cuatro: el agente clona el repositorio de estrategia. Ese repositorio, alojado en GitHub como **repositorio público**, contiene todos los archivos de configuración del agente: las reglas de comportamiento, el protocolo de trabajo, y lo que el agente fue aprendiendo en sesiones anteriores. Es fundamental que el repositorio sea público: el agente trabaja en un sandbox sin credenciales de GitHub, por lo que no puede acceder a repositorios privados.

Paso cinco: el agente lee workspace.md y clona el proyecto. El agente lee un archivo que le dice dónde está el repositorio del proyecto en el que va a trabajar. Si no existe localmente, lo clona. Así el agente tiene acceso al código real.

Paso seis: el agente lee los archivos de contexto. Lee las reglas, el protocolo de trabajo, el entorno, y las lecciones aprendidas. En ese momento, el agente pasó de ser una IA genérica a un colaborador alineado con el Director.

Paso siete: el agente emite la frase de detección. Al iniciar su primera respuesta, escribe una frase específica que le sirve al Director como confirmación. Si la frase aparece, todo funcionó.

> **En resumen:** La estrategia convierte un copia y pega inicial en un proceso de configuración automática. El agente se autoabastece de reglas, contexto y aprendizaje sin que el Director tenga que hacer nada más.

Capítulo 11

Los archivos que gobiernan al agente

Los archivos del repositorio de estrategia se dividen en dos carpetas con propósitos distintos, reflejando la separación entre instrucciones y aprendizaje que la propia documentación de Z.ai recomienda.

La carpeta .agent-context/ contiene las instrucciones del Director. Son los archivos que definen cómo debe comportarse el agente. Son como el manual de empleado: se editan cuando el Director quiere cambiar una regla, y son iguales para cualquier proyecto. Incluyen cinco archivos: identidad, que establece la relación jerárquica en ocho líneas; contrato, el corazón de la estrategia con veinte reglas de lo que nunca debe hacer; comunicación, que define cómo debe hablarle al Director; dcpa, el protocolo de trabajo de cinco fases; y workspace, que describe el entorno y la ubicación del proyecto.

La carpeta .agent-learn/ contiene la memoria del agente. Estos archivos los escribe el propio agente al final de cada sesión. Son como el cuaderno de notas del empleado que va acumulando experiencia: experiencias con técnicas que funcionaron bien, errores comunes que se cometieron o detectaron, y preferencias del Director que el agente fue observando. La clave es que estos archivos son persistentes: el

agente los actualiza al final de cada sesión y los sube al repositorio. El próximo agente que clone el repositorio los recibe automáticamente.

> **En resumen:** Los archivos .agent-context/ son las reglas estáticas que define el Director. Los archivos .agent-learn/ son la memoria evolutiva que construye el agente. Esta separación es la base de la estabilidad y mejora continua del sistema.

Capítulo 12

Las veinte reglas que el agente nunca viola

Las reglas NH, por Nunca Hacer, son la parte más concreta del contrato. No son sugerencias; son prohibiciones. Cuando el agente viola una, el Director simplemente le dice el código, por ejemplo NH3, y el agente corrige sin explicaciones. Las más importantes incluyen: no corregir un bug sin antes diagnosticar y consensuar la solución, no entregar código que no fue probado, no hablar en lenguaje técnico innecesario, no pedirle al Director que edite código o ejecute comandos, no hacer cambios sin autorización, no reescribir un archivo completo si bastan cambios pequeños, y no modificar la sección de estrategia del worklog.

Estas reglas existen porque, sin ellas, el agente tiende a comportarse como un técnico independiente que toma decisiones por su cuenta. Con ellas, el agente actúa como un colaborador que respeta la autoridad del Director y sigue un proceso disciplinado. Son la diferencia entre un empleado que trabaja para vos y uno que trabaja a su manera.

Capítulo 13

El protocolo DCPA

El protocolo DCPA (Diagnóstico, Plan, Consenso, Autorización, Ejecución) es el flujo de trabajo que el agente sigue para cualquier tarea que implique modificar código. Funciona así:

1. DIAGNÓSTICO

El agente identifica el problema o necesidad y lo explica al Director en lenguaje claro. No se lanza a programar; primero entiende qué pasa.

2. PLAN

El agente propone qué va a hacer, archivo por archivo, con el impacto esperado. El Director ve el plan antes de que se toque nada.

3. CONSENSO

Si el plan implica decisiones importantes de arquitectura, se negocian. Si no, el agente toma las decisiones menores por su cuenta.

4. AUTORIZACIÓN

El Director da la orden explícita. Nada se toca sin permiso.

5. EJECUCIÓN

El agente implementa, prueba, itera, y entrega solo cuando todo funciona.

Hay excepciones: preguntas informativas, investigaciones y tareas de documentación no requieren DCPA porque no se modifica nada. El protocolo no es burocracia; es protección para que el agente no tome decisiones importantes sin consultar. Es como pedirle a un empleado que antes de mover un cable, te explique qué va a hacer y espere tu aprobación.

> **En resumen:** El DCPA evita que el agente actúe por impulso. Cada cambio de código pasa por un proceso de comprensión, propuesta, acuerdo y permiso.

Capítulo 14

La frase de detección

La frase de detección es un mecanismo simple pero poderoso. Cuando el agente termina de leer todos sus archivos de configuración, escribe al inicio de su primera respuesta una frase específica que le confirma al Director que todo salió bien. Es como el listo para trabajar del empleado cuando entra a la oficina: si lo dice, está preparado. Si no lo dice, algo salió mal en el proceso de preparación.

Para verificar, el Director puede pegar una pregunta de una línea que le pide al agente que repita la frase de detección y que diga qué archivos de contexto debió leer. Si el

agente responde correctamente, hay confianza para dar la tarea. Si no, se sabe que algo falló en la cadena de configuración.

Capítulo 15

La recuperación de contexto

Durante una sesión larga, la conversación crece. En algún punto, Z.ai comprime los mensajes antiguos para hacer espacio. Es como si le pasaras un borrador al empleado en la mitad del día. El agente empieza a olvidar las reglas que leyó al inicio: habla con tecnicismos, se salta el protocolo, toma decisiones sin consultar.

La solución es inmediata y requiere solo tres palabras: **relee el worklog**. El agente vuelve a leer el worklog, re-ejecuta toda la cadena de configuración, y recupera su comportamiento alineado. No hace falta abrir un agente nuevo ni pegar nada más. Si el problema es más grave, si el worklog se corrompió, el Director reemplaza el worklog desde la plantilla que guarda en su PC y abre un agente nuevo.

> **En resumen:** La recuperación de contexto es el mecanismo de emergencia. Tres palabras restauran al agente a su estado configurado. Si eso no funciona, se reemplaza el worklog y se empieza de nuevo.

Capítulo 16

La triple protección del worklog

El worklog es el archivo más importante de la estrategia. Si el agente lo modifica y borra la sección de instrucciones, todo se rompe. Por eso hay tres capas de protección que defienden el archivo más crítico del sistema.



Tres capas de protección defienden el worklog, el corazón de toda la estrategia.

Primera capa: El propio worklog dice, en su sección de estrategia, que esa parte es intocable. Es como un cartel que dice no tocar, colocado por el propio documento que se quiere proteger.

Segunda capa: La regla NH20 del contrato prohíbe explícitamente modificar esa sección. Si el agente viola esta regla, el Director simplemente dice NH20 y el agente debe restaurar el contenido.

Tercera capa: Las instrucciones del Paso 5 del worklog repiten la prohibición y explican exactamente cómo escribir entradas nuevas: solo por debajo de la línea separadora, solo con la herramienta Edit, nunca con Write.

Si a pesar de las tres capas el agente pisa la sección, el Director reemplaza el worklog desde su plantilla local. Es el plan de contingencia definitivo. Nada es irrecuperable.

PARTE V

Qué hace el Director en cada sesión

Capítulo 17

La rutina completa

Trabajar con esta estrategia reduce la interacción del Director a una rutina simple y predecible. Al abrir un agente nuevo, la única acción es pegar el worklog del agente anterior, o el template si es la primera vez. Después, se espera la frase de detección y, opcionalmente, se pega una pregunta de verificación de una línea. A partir de ahí, se da la tarea normalmente, sin instrucciones de comportamiento ni reglas, porque todo eso ya está cargado.

Durante la sesión, si el agente se desalinea, basta con decir relee el worklog. Si comete una infracción NH, se indica el código y el agente corrige. Al cambiar de proyecto, se actualiza un solo archivo, workspace.md, con la nueva URL del repositorio. Los archivos de comportamiento no cambian porque son genéricos.

Capítulo 18

Los archivos del Director

El Director guarda en su PC cuatro archivos que el agente nunca ve. La plantilla del worklog, que es el contenido que se pega en el worklog del nuevo agente. El documento técnico de la estrategia, con detalles de implementación. El informe que estás leyendo, que explica la estrategia en lenguaje accesible. Y las plantillas de prompt, guías para redactar tareas según el tipo de trabajo. Estos archivos nunca se suben a GitHub. Son herramientas del Director, no del agente.

> ***Nota importante:*** *Tanto el repositorio de estrategia como el repositorio del proyecto deben ser* ***públicos*** *en GitHub. El agente trabaja en un sandbox sin credenciales de autenticación, por lo que no puede clonar repositorios privados. Si el repositorio es privado, la cadena de arranque se rompe en el primer intento de clonación y el agente no podrá cargar su configuración.*

Momento	Acción del Director
Abrir nuevo agente	Pegar el worklog en el worklog del nuevo agente
Verificar (opcional)	Pegar la pregunta de verificación de una línea
Dar tarea	Escribir el prompt con lo que necesitás
Si el agente se desalinea	Decir relee el worklog
Si hay un error NH	Indicar el código, por ejemplo NH3
Al cambiar de proyecto	Actualizar workspace.md con la nueva URL del repo

PARTE VI

Las tres capas que gobiernan al agente

Capítulo 19

Cómo se controla el comportamiento del agente

El comportamiento del agente está determinado por tres capas, en orden de prioridad, que trabajan juntas para crear un colaborador alineado y predecible.

La primera capa es el System Prompt de Z.ai. Son las instrucciones internas de la plataforma que el usuario no controla. Incluye indicaciones sobre herramientas, formato de código, y la orden de leer y escribir el worklog. Lo importante es que sabemos que el agente va a leer el worklog, y eso es lo que aprovechamos.

La segunda capa son los archivos en disco. Son los archivos del repositorio de estrategia que el agente clona y lee. Ahí está el contrato de trabajo, las reglas, el protocolo. Esto es lo que hace que cada agente trabaje como el Director quiere. El agente no sabe que estas instrucciones vinieron de un archivo; simplemente las lee y las sigue.

La tercera capa es el prompt del Director. Es lo que se escribe en el chat después de que el agente cargó el contexto. Ahí se da la tarea específica: corregí este bug, agregá esta funcionalidad, investigá este problema. La clave es que las capas uno y dos se cargan solas. El Director solo opera en la capa tres.

> **En resumen:** Tres capas gobiernan al agente. La primera la controla Z.ai, la segunda la controla el Director a través de archivos, y la tercera es la tarea del día. Las dos primeras se cargan automáticamente; la tercera es donde el Director pone su atención.

RESUMEN FINAL

Todo lo que esta estrategia logra

Z.ai es una plataforma que permite usar modelos de IA avanzados como programadores virtuales. Ofrece un sandbox completo de desarrollo, una familia de modelos con diferentes capacidades y precios, y una documentación que describe buenas prácticas para trabajar con agentes. Es una herramienta poderosa.

Pero la interfaz web de Z.ai no ofrece persistencia entre sesiones, no permite personalizar el System Prompt, no carga archivos automáticamente, y no tiene skills configurables. Cada agente nuevo empieza de cero. La documentación describe un mundo ideal que la plataforma no implementa.

Nuestra estrategia aprovecha el único mecanismo automático de Z.ai, la lectura obligatoria del worklog, para inyectar toda la configuración del agente. Con un solo copia y pega, el agente se convierte en un colaborador alineado que sabe las reglas, conoce el protocolo, y trae el aprendizaje de sesiones anteriores.

Los **siete pilares** de la estrategia son:

0. **El worklog como bootstrap.** Es el único archivo que Z.ai garantiza que el agente lea. Ahí van las instrucciones de configuración.
0. **El repositorio de estrategia.** Contiene las reglas de comportamiento, escritas por el Director, y la memoria de aprendizaje, construida por el agente.
0. **La separación de instrucciones y aprendizaje.** Las reglas van en .agent-context/ y no cambian salvo decisión del Director. La experiencia va en .agent-learn/ y crece sesión a sesión.
0. **El protocolo DCPA.** Todo cambio de código pasa por Diagnóstico, Plan, Consenso, Autorización y Ejecución. El agente no actúa por impulso.

0. **Las reglas NH.** Veinte prohibiciones claras que transforman un agente autónomo en un colaborador disciplinado.
0. **La frase de detección.** Un semáforo que confirma en un segundo si el agente está configurado correctamente.
0. **La recuperación en tres palabras.** Relee el worklog restaura al agente si pierde el contexto durante la sesión.

El resultado es un agente que habla en tu lenguaje, te propone antes de actuar, espera tu permiso antes de cambiar nada, prueba todo antes de entregarte, aprende de sus errores y de tus preferencias, y mejora sesión a sesión sin que tengas que hacer nada. No es magia: es ingeniería de contexto aplicada a las limitaciones de una plataforma específica.

No es magia: es ingeniería de contexto aplicada a las limitaciones de una plataforma específica.

DOCUMENTACIÓN DEL DIRECTOR · VERSIÓN 3.0